

Yesterday 5:38 PM

diff between them

```
//int? x = null;  
////Nullable<int> y = null;  
//int x = 3;
```

The difference is **whether the `int` variable is allowed to contain `null`** .

1. `int? x = null;`

```
</> C#
```

```
int? x = null;
```

`int?` means **nullable int**.

It is shorthand for:

```
</> C#
```

```
Nullable<int> x = null;
```

So these two are exactly equivalent:

```
</> C#
```

```
int? x = null;  
Nullable<int> y = null;
```

The variable can contain either:

```
</> Plain text
```

```
an integer value → 5  
null → no value
```

Example:

```
</> C#
```

```
int? age = null;
```

```
age = 25;    // valid
age = null;  // valid
```

2. `Nullable<int> y = null;`

```
</> C#
Nullable<int> y = null;
```

This is the **long form** of `int?` .

So:

```
</> C#
int? x
```

and

```
</> C#
Nullable<int> x
```

mean the same thing.

Usually developers use `int?` because it's shorter and easier to read.

3. `int x = 3;`

```
</> C#
int x = 3;
```

This is a **normal non-nullable integer**.

It must always contain an integer value.

```
</> C#
```

```
int x = 3;    // ✓
x = 10;      // ✓
x = 0;       // ✓
x = null;    // ✗
```

Think of it like this:

Declaration	Can hold number?	Can hold null ?
<code>int x</code>	✓	✗
<code>int? x</code>	✓	✓
<code>Nullable<int> x</code>	✓	✓

Important: `int?` does **not** mean the object/value is useless when it's `null`. It means the variable represents an integer **that may or may not currently have a value**.

This is especially common with database columns where a value is optional, e.g.:

```
</> C#
int? age
DateTime? birthDate
decimal? discount
```

Because the database may contain `NULL`.

implicit casting and explicit casting

In C#, **casting** means converting a value from one type to another.

The main difference is **whether C# can safely perform the conversion automatically**.

1. Implicit casting

No code is required from you. C# automatically converts the type.

Usually this happens when converting from a **smaller range type** → **larger range type**, because there is no risk of losing information.

```
</> C#
int x = 10;
long y = x;
```

Here:

</> Plain text

```
int → long
```

`long` can represent every possible `int`, so C# does it automatically.

Another example:

</> C#

```
int x = 10;  
double y = x;
```

No cast needed.

Common examples

</> Plain text

```
byte → short → int → long → float → double
```

For example:

</> C#

```
int x = 100;  
double y = x; // implicit
```

2. Explicit casting

Here, you must tell C# to perform the conversion using `(type)`.

Usually required when there is a possibility of **data loss**.

</> C#

```
double x = 10.5;  
int y = (int)x;
```

The `(int)` is the explicit cast.

Result:

</> Plain text

```
x = 10.5  
y = 10
```

The decimal part is lost.

Another example:

</> C#

```
long x = 100;  
int y = (int)x;
```

You explicitly tell C#:

```
"Convert this long to an int ."
```

Why doesn't C# do this automatically?

Consider:

</> C#

```
long x = 5_000_000_000;  
int y = (int)x;
```

int cannot hold 5,000,000,000 , so converting can cause **overflow/data loss**.

That's why C# requires:

</> C#

```
int y = (int)x;
```

You're explicitly accepting responsibility for the conversion.

Easy interview rule

Implicit = safe conversion → C# does it automatically.

Explicit = potentially unsafe/lossy conversion → you use (type) .

Important connection to your previous int? question

Nullable types add another situation:

```
</> C#  
  
int? x = 10;  
int y = x;      // ❌
```

Why? Because `x` might be null.

You can explicitly access the value:

```
</> C#  
  
int y = (int)x;
```

or safely handle the null:

```
</> C#  
  
int y = x ?? 0;
```

So don't confuse **casting** with **nullable value handling**—they are related but different concepts.

1 selection

it throws exception or what?

Yes—but **not necessarily an exception** in this exact code.

```
</> C#  
  
long x = 5_000_000_000;  
int y = (int)x;
```

By default, an explicit numeric cast from `long` to `int` uses **unchecked conversion**.

Since `5,000,000,000` is larger than `int.MaxValue` (`2,147,483,647`), the value **overflows** and `y` gets an incorrect wrapped value.

```
</> C#
```

```
Console.WriteLine(y);
```

You won't get an exception by default.

When does it throw?

If you use `checked` :

```
</> C#
```

```
long x = 5_000_000_000;  
int y = checked((int)x);
```

Now C# detects the overflow and throws:

```
</> Plain text
```

```
System.OverflowException
```

So:

Code	Result
<code>(int)x</code>	 Overflow, usually no exception
<code>checked((int)x)</code>	 <code>OverflowException</code>

This is an important distinction for C# interviews: **explicit casting does not automatically mean an exception will be thrown.**

 1 selection

what it is printed here

For:

```
</> C#
```

```
long x = 5_000_000_000;  
int y = (int)x;  
  
Console.WriteLine(y);
```

It prints:

</> Plain text

```
705032704
```

Why? `int` can only hold values from:

</> Plain text

```
-2,147,483,648
```

```
to
```

```
2,147,483,647
```

The explicit cast is **unchecked**, so the bits are truncated/wrapped around rather than throwing an exception.

So:

</> Plain text

```
5,000,000,000L
```

```
    ↓ (int)
```

```
705,032,704
```

If you use:

</> C#

```
int y = checked((int)x);
```

then you get an **OverflowException** instead of a value.

1 selection

why this number why not 2,147,483,647

Because `(int)` doesn't clamp the value to `int.MaxValue`.

It converts the number using the **lower 32 bits** of the `long` value.

Step by step

Your value is:

</> Plain text

```
long x = 5,000,000,000
```

An `int` has exactly 32 bits.

5,000,000,000 in binary needs more than 32 bits, so when you explicitly cast to `int`, only the lower 32 bits are kept.

Mathematically, the result is:

</> Plain text

$$5,000,000,000 - 2^{32}$$

Since:

</> Plain text

$$2^{32} = 4,294,967,296$$

we get:

</> Plain text

```
5,000,000,000
-4,294,967,296
-----
 705,032,704
```

So:

</> C#

```
long x = 5_000_000_000;
int y = (int)x;

Console.WriteLine(y);
```

prints:

</> Plain text

```
705032704
```

Why not 2,147,483,647 ?

Because 2,147,483,647 is simply the **maximum value** an `int` can represent.

C# doesn't say:

"The number is too big, so I'll use the maximum."

Instead, an unchecked numeric conversion effectively **wraps around modulo 2^{32}** .

If you actually want to prevent this overflow, use:

</> C#

```
int y = checked((int)x);
```

Then you'll get:

</> Plain text

OverflowException

Interview tip:

`(int)5_000_000_000` → **overflow/wraparound**

`checked((int)5_000_000_000)` → **OverflowException**

can `int` be implicitly casted to `float`

Yes. ☒ `int` can be **implicitly converted** to `float` in C#.

</> C#

```
int x = 10;  
float y = x;
```

No explicit cast is needed.

Why?

Because `float` has a much larger **range** than `int` :

</> Plain text

`int` → approximately ± 2.1 billion

`float` → approximately $\pm 3.4 \times 10^{38}$

So C# allows:

</> Plain text

`int` → `float` ☒ implicit

However, there's an important detail: `float` does not have enough precision to represent every possible `int` exactly.

For example, large integers can lose precision when converted to `float` :

</> C#

```
int x = 16_777_217;  
float y = x;  
  
Console.WriteLine(y);
```

The `float` may represent this as 16,777,216 because a `float` has only about 7 decimal digits of precision.

So remember:

Implicit conversion doesn't necessarily mean "no information can ever be lost." It means C# considers the conversion allowed without requiring an explicit cast.

 1 selection

but how both has 32 bits

Exactly — **both are 32 bits**, but the bits are used differently.

The key is:

Number of bits $\neq$ range + precision in the same way.

`int` — 32 bits

An `int` uses all 32 bits to represent an **integer exactly**.

</> Plain text

```
int
[ sign + 31 other bits ]
```

Range:

</> Plain text

-2,147,483,648 → 2,147,483,647

So it can represent about **4.3 billion different integer values**.

float — also 32 bits

A `float` uses its 32 bits differently:

</> Plain text

```
[ sign | exponent | fraction ]
  1      8      23
```

So it sacrifices **precision** to get a huge **range**.

Conceptually:

</> Plain text

$\text{float} = \text{sign} \times \text{significand} \times 2^{\text{exponent}}$

The **8-bit exponent** allows the number to become extremely large:

</> Plain text

$\approx 3.4 \times 10^{38}$

But because only **23 bits + an implicit bit** are used for the significant digits, it cannot represent every integer exactly.

Think of it this way

Imagine you have **32 boxes**.

For `int` :

</> Plain text

32 boxes → all dedicated to representing the exact integer

For `float` :

</> Plain text

1 box → sign

8 boxes → exponent

23 boxes → precision

So:

	<code>int</code>	<code>float</code>
Size	32 bits	32 bits
Huge range	✗	✓
Every integer exactly	✓ within its range	✗
Decimal numbers	✗	✓
Approx. max	2.1×10^9	3.4×10^{38}

The important concept

`int` gives you **precision**.

`float` gives you **range** at the cost of **precision**.

That's why this is possible:

</> C#

```
int x = 2_000_000_000;
float y = x; // implicit
```

Even though `float` can represent numbers much larger than `int` , it **cannot represent every number between them exactly**.

so double and decimal are not good in pericsion like float?

Actually, **double** and **decimal** are more precise than **float** , but in different ways.

1. float

32 bits

- ~7 significant decimal digits
- Very large range
- Lower precision

</> C#

```
float x = 1.123456789f;  
// roughly 1.123457
```

2. double

64 bits

- ~15–16 significant decimal digits
- Much more precision than **float**
- Very large range

</> C#

```
double x = 1.123456789123456789;  
// roughly 1.1234567891234567
```

3. decimal

128 bits

- ~28–29 significant decimal digits
- Designed for **high-precision decimal arithmetic**
- Smaller range than **double**
- Common for **money/financial calculations**

</> C#

```
decimal price = 10.25m;
```

Quick comparison

Type	Size	Precision	Common use
float	32-bit	~7 digits	Graphics, games
double	64-bit	~15–16 digits	Scientific/general calculations
decimal	128-bit	~28–29 digits	Money/financial
int	32-bit	Exact integers	Counting

So the general idea is:

</> Plain text

```
float    → least precision
double   → much more precision
decimal  → highest decimal precision
```

One important interview point: **decimal** isn't simply "more precise **double** ." It uses a different representation and is particularly useful when exact decimal arithmetic matters, such as prices and financial values.

what do u mean by significant decimal digits

Significant decimal digits means the number of digits that a numeric type can reliably keep **with meaningful precision**.

Think of it as the number of **important digits**, starting from the first non-zero digit.

Example

Suppose:

</> Plain text

```
123.456789
```

This has **9 significant digits**:

</> Plain text

```
123456789
^^^^^^^^^
9 significant digits
```

If a type can store about **7 significant digits**, like `float`, it can only reliably preserve something like:

</> Plain text

```
123.4567
```

The remaining digits are approximated.

Another example

Consider:

</> Plain text

```
123456789
```

There are **9 significant digits**.

A `float` has about **7 significant decimal digits**, so it cannot represent this exact integer.

You might get something like:

</> Plain text

```
123456792
```

The exact result depends on the binary representation.

What about zeros?

Leading zeros usually **don't count**:

</> Plain text

```
0.00012345
```

The significant digits are:

</> Plain text

```
12345
```

```
^^^^
```

```
5 significant digits
```

But zeros between or after meaningful digits can count:

```
</> Plain text
```

```
120.05
```

```
^^^^
```

```
5 significant digits
```

Why does this matter for `float` ?

`float` has approximately **7 significant digits**.

So:

```
</> Plain text
```

```
1234567    ← 7 digits → can be represented with reasonable precision
```

```
12345678    ← 8 digits → precision starts being lost
```

And this is why:

```
</> C#
```

```
float x = 16_777_217;
```

can become:

```
</> Plain text
```

```
16_777_216
```

Even though `float` has a huge **range**, it doesn't have enough **precision** to distinguish every integer at that magnitude.

Simple way to remember

Range = *How big/small can the number be?*

Precision = *How many meaningful digits can I accurately represent?*

That's the key difference between `float` , `double` , and `decimal` .

explain all things related to structs

Absolutely. Since you're preparing for **C#/.NET interviews**, let's cover structs from the interview perspective, from the basics to the tricky parts.

1. What is a struct ?

A **struct** is a **value type** in C# that allows you to create your own type containing data and behavior.

</> C#

```
struct Point
{
    public int X;
    public int Y;
}
```

You can use it:

</> C#

```
Point p = new Point();

p.X = 10;
p.Y = 20;
```

Think of a struct as a **small custom value type**.

2. Struct vs Class

This is the most important distinction.

</> C#

```
struct Point
{
    public int X;
    public int Y;
}
```

versus:

```
</> C#  
  
class Person  
{  
    public string Name;  
}
```

The fundamental difference:

	struct	class
Type	Value type	Reference type
Assignment	Copies the value	Copies the reference
Can be null ?	✗ normally	✓
Inheritance	✗ cannot inherit from class/struct	✓
Can implement interfaces?	✓	✓
Default value	Zero/default values	null reference
Usually used for	Small values	Objects/entities

3. Value semantics

This is probably the most important concept.

Consider:

```
</> C#  
  
struct Point  
{  
    public int X;  
}
```

Then:

```
</> C#  
  
Point p1 = new Point();  
p1.X = 10;
```

```
Point p2 = p1;
```

```
p2.X = 20;
```

What is:

```
</> C#
```

```
p1.X
```

?

10.

Why?

Because:

```
</> C#
```

```
Point p2 = p1;
```

creates a **copy** of the value.

Conceptually:

```
</> Plain text
```

```
p1
```

X=10

```
p2
```

X=10

Changing p2 doesn't change p1 .

4. Compare with a class

Now:

</> C#

```
class Point
{
    public int X;
}
```

And:

</> C#

```
Point p1 = new Point();
p1.X = 10;

Point p2 = p1;

p2.X = 20;
```

Now:

</> C#

```
Console.WriteLine(p1.X);
```

prints:

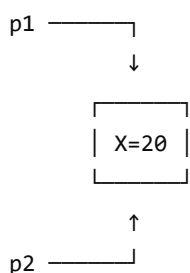
</> Plain text

20

Because both variables refer to the **same object**.

Conceptually:

</> Plain text



So:

Struct assignment copies the value.

Class assignment copies the reference.

5. Where is a struct stored?

This is where many interview explanations become misleading.

You may hear:

"Structs are stored on the stack and classes are stored on the heap."

That's **not always true**.

A struct is a **value type**, but its actual storage depends on where it exists.

For example:

```
</> C#
```

```
Point p = new Point();
```

A local struct variable can commonly live in stack-like local storage.

But:

```
</> C#
```

```
class Person
{
    public Point Location;
}
```

Here `Location` is part of the `Person` object, so the struct is stored **inside the object on the heap**.

Therefore:

Value type $\neq$ always stack.

Reference type $\neq$ simply "heap variable".

The important concept is **value semantics**, not stack vs heap.

6. Structs cannot inherit from classes

You cannot do:

</> C#

```
struct Point : SomeClass
{
}
```

 Not allowed.

Structs implicitly derive from:

</> Plain text

```
System.ValueType
```

which itself derives from:

</> Plain text

```
System.Object
```

Conceptually:

</> Plain text

```
System.Object
  ↓
System.ValueType
  ↓
struct
```

But you can implement interfaces:

</> C#

```
struct Point : IComparable<Point>
{
    // implementation
}
```

 Allowed.

7. Can a struct contain methods?

Absolutely.

A struct isn't just a collection of fields.

</> C#

```
struct Point
{
    public int X;
    public int Y;

    public void Print()
    {
        Console.WriteLine($"{X}, {Y}");
    }
}
```

Then:

</> C#

```
Point p = new Point();

p.X = 10;
p.Y = 20;

p.Print();
```

8. Struct constructors

Modern C# allows structs to have constructors.

</> C#

```
struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

```
}  
}
```

Then:

</> C#

```
Point p = new Point(10, 20);
```

9. Default value of a struct

Every struct has a **default value**.

For:

</> C#

```
struct Point  
{  
    public int X;  
    public int Y;  
}
```

you can do:

</> C#

```
Point p = default;
```

Then:

</> Plain text

```
X = 0  
Y = 0
```

because the default value of `int` is `0`.

You can also write:

</> C#

```
Point p = new Point();
```

which produces the default struct value.

10. Can a struct be null?

Normally:

```
</> C#
```

```
Point p = null;
```

✗ Not allowed.

Because structs are value types.

But you can make a **nullable struct**:

```
</> C#
```

```
Point? p = null;
```

This is shorthand for:

```
</> C#
```

```
Nullable<Point> p = null;
```

This is exactly the same concept you were asking about with:

```
</> C#
```

```
int?
```

Remember:

```
</> Plain text
```

```
int?      → Nullable<int>
```

```
Point?    → Nullable<Point>
```

```
DateTime? → Nullable<DateTime>
```

11. Boxing

This is an important interview topic.

Because structs are value types, they can be **boxed** into an object.

</> C#

```
int x = 10;
```

```
object obj = x;
```

Here the `int` value is placed inside an object.

Conceptually:

</> Plain text

```
int x
```

```
↓
```

```
boxing
```

```
↓
```

```
object
```

The same applies to your own struct:

</> C#

```
Point p = new Point();
```

```
object obj = p;
```

That's **boxing**.

12. Unboxing

Going back from `object` to the value type is called **unboxing**.

</> C#

```
object obj = 10;
```

```
int x = (int)obj;
```

This requires an explicit cast.

Similarly:

```
</> C#
```

```
Point p = (Point)obj;
```

if `obj` actually contains a boxed `Point` .

13. Structs and Equals

Structs inherit behavior from `ValueType` .

For example:

```
</> C#
```

```
Point p1 = new Point { X = 10, Y = 20 };  
Point p2 = new Point { X = 10, Y = 20 };
```

When comparing values, structs are designed around **value equality**.

Conceptually:

```
</> Plain text
```

```
p1 = (10,20)  
p2 = (10,20)  
  
p1 == p2
```

Whether `==` itself is available depends on whether the struct defines an appropriate `==` operator. Otherwise, use/override equality appropriately.

For your own structs, you should understand:

- `Equals()`
- `GetHashCode()`
- `IEquatable<T>`

especially if the struct will be used as a dictionary key.

14. Why shouldn't structs be huge?

Because structs have **value semantics**.

Suppose:

```
</> C#  
  
struct HugeStruct  
{  
    // lots of fields...  
}
```

And you do:

```
</> C#  
  
HugeStruct a = ...;  
HugeStruct b = a;
```

The entire value may need to be copied.

With a class:

```
</> C#  
  
HugeClass b = a;
```

you're generally just copying a reference.

Therefore, structs are generally best suited to **small, lightweight values**.

Examples:

```
</> Plain text  
  
Point  
DateTime  
TimeSpan  
Guid  
decimal
```

These are all structs.

15. Structs are useful for representing a value

This is the key design idea.

A Point :

</> Plain text

X = 10

Y = 20

is naturally a **value**.

A Person :

</> Plain text

Name

Address

Phone

...

is usually treated as an **object/entity** with identity.

So:

</> Plain text

Point → value

Money → value

Date → value

Temperature → value

Person → object

Order → object

Customer → object

Product → object

This isn't an absolute rule, but it's a useful design guideline.

16. Readonly structs

C# supports:

</> C#

```
readonly struct Point
{
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

A `readonly struct` is intended to be immutable.

Meaning:

Once created, its state shouldn't change.

This can improve safety and sometimes performance because the compiler/runtime can avoid certain defensive copies.

17. ref struct

This is a more advanced topic.

You can have:

</> C#

```
ref struct MyStruct
{
}
```

`ref struct` has special restrictions because it is designed for scenarios involving **stack-only/ref-like data**.

A famous example is:

</> C#

Span<T>

Span<T> is a `ref struct` .

Because of its restrictions, a `ref struct` cannot generally:

- be boxed
- be stored as a field of a normal class
- cross certain `async / await` boundaries
- be used where a heap object might be required

You don't need to memorize all restrictions initially. Just remember:

`ref struct` is an advanced stack-only/ref-like struct used for high-performance scenarios such as `Span<T>` .

18. Struct fields and reference types

A struct can contain reference-type fields.

For example:

</> C#

```
struct PersonInfo
{
    public string Name;
}
```

The struct itself is a value type, but `Name` is a reference to a string.

So don't think:

"Everything inside a struct must be stored directly."

A struct can contain references.

19. Struct passed to a method

By default, structs are passed **by value**.

</> C#

```
void Change(Point p)
{
    p.X = 100;
}
```

If:

</> C#

```
Point p = new Point();
p.X = 10;

Change(p);
```

the original `p.X` remains:

</> Plain text

10

because the method received a **copy**.

20. Using `ref`

You can explicitly pass the struct by reference:

</> C#

```
void Change(ref Point p)
{
    p.X = 100;
}
```

Then:

</> C#

```
Change(ref p);
```

Now the original struct is modified.

So:

</> Plain text

normal parameter

struct → copy

ref parameter

struct → reference to original

21. Mutable vs immutable structs

A mutable struct allows:

</> C#

```
p.X = 10;
```

```
p.X = 20;
```

An immutable struct doesn't allow its state to change after creation.

For example, conceptually:

</> C#

```
readonly struct Point
```

```
{
```

```
    public int X { get; }
```

```
    public int Y { get; }
```

```
}
```

For most custom structs, **immutability is generally preferred** because mutable structs can create confusing behavior due to copying.

22. The biggest interview traps

Trap 1

"Structs are always stored on the stack."

✗ Incorrect.

Structs are **value types**, but their physical storage depends on context.

Trap 2

"Structs cannot have methods."

✗ Incorrect.

They can have:

- fields
 - properties
 - methods
 - constructors
 - operators
 - events
 - interfaces
-

Trap 3

"Structs cannot implement interfaces."

✗ Incorrect.

They absolutely can.

</> C#

```
struct MyStruct : IDisposable
{
    public void Dispose()
    {
    }
}
```

Trap 4

"A struct can never be null."

Not exactly.

```
</> C#
```

```
Point p = null;
```



But:

```
</> C#
```

```
Point? p = null;
```

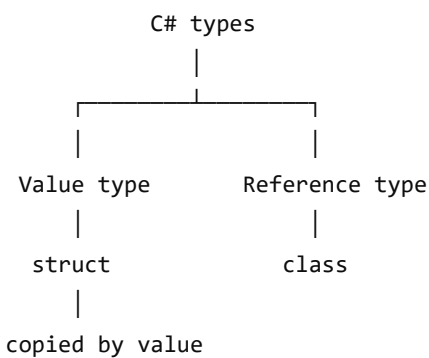


The mental model I'd recommend

Don't memorize "struct = stack, class = heap."

Instead remember:

```
</> Plain text
```



So when you see:

```
</> C#
```

```
Point p2 = p1;
```

immediately think:

A new copy of the struct's value is created.

Whereas:

```
</> C#
```

```
Person p2 = p1;
```

means:

Both variables refer to the same object.

That **value semantics vs reference semantics** distinction is the most important thing to understand about structs.

how struct is value type and has ctor and new()

Yes — this seems contradictory at first, but the key is:

new does NOT mean "reference type" and a constructor does NOT mean "class".

A `struct` is still a **value type**, even when you use `new`.

1. Without `new`

Consider:

```
</> C#
```

```
struct Point
{
    public int X;
    public int Y;
}
```

```
Point p;
```

`p` is a **value-type variable**.

You can think of it as having space for:

```
</> Plain text
```

```
p
```

X
Y

It isn't a reference pointing to a `Point` object.

2. What does `new Point()` mean?

`</> C#`

```
Point p = new Point();
```

Here `new Point()` means:

Create/initialize a `Point` value using its constructor.

It does **not** mean:

"Put a `Point` object on the heap and make `p` a reference."

That's the important distinction.

Compare:

`</> C#`

```
class Person  
{  
}
```

```
Person p = new Person();
```

Here:

`</> Plain text`

`p` $\longrightarrow$ `Person` object

`p` is a reference to an object.

But:

`</> C#`

```
struct Point  
{  
    public int X;  
    public int Y;  
}
```

```
Point p = new Point();
```

conceptually:

</> Plain text

p

X = 0
Y = 0

p itself contains the value.

3. Why does a struct need a constructor?

Because a struct can have initialization logic.

For example:

</> C#

```
struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Then:

</> C#

```
Point p = new Point(10, 20);
```

The constructor initializes the value:

</> Plain text

p

X = 10

```
| Y = 20 |
```

It's still a value type.

4. This is the important part

`new` has two different-looking effects depending on the type.

For a class:

```
</> C#
```

```
Person p = new Person();
```

`new` creates a class object and gives you a reference to it.

For a struct:

```
</> C#
```

```
Point p = new Point();
```

`new` creates/initializes a **value**.

So:

```
</> Plain text
```

```
class + new
```

↓

```
object + reference
```

```
struct + new
```

↓

```
value
```

5. Then why call it a constructor?

A constructor is simply a mechanism for **initializing an instance/value**.

It isn't exclusive to classes.

Both can have constructors:

```
</> C#  
  
class Person  
{  
    public Person(string name) { }  
}
```

and:

```
</> C#  
  
struct Point  
{  
    public Point(int x, int y) { }  
}
```

The difference is the **type semantics**, not whether a constructor exists.

6. One more important thing

You might hear:

" new always allocates on the heap."

That's also **not correct**.

`new` is an operator used to create/initialize an instance. Whether an actual heap allocation occurs depends on the type and context.

For example:

```
</> C#  
  
int x = new int();
```

`int` is a struct/value type.

And:

```
</> C#  
  
DateTime date = new DateTime();
```

`DateTime` is also a struct.

Neither suddenly becomes a reference type just because `new` was used.

The rule to remember

Value type vs reference type is determined by the type definition, not by whether you use `new`.

`struct` → value type

`class` → reference type

if i want to create struct instance does i have to use new especially
if there is no ctor

No. **You don't always have to use `new` with a struct.** This is one of the differences between structs and classes.

Suppose:

```
</> C#
```

```
struct Point
{
    public int X;
    public int Y;
}
```

1. You can declare it without `new`

```
</> C#
```

```
Point p;
```

This creates a **struct variable**, but it is **not definitely assigned yet**.

You cannot do:

```
</> C#
```

```
Console.WriteLine(p.X); // ❌
```

because `p` hasn't been initialized.

But you can assign all its fields:

```
</> C#
```

```
p.X = 10;  
p.Y = 20;
```

Now it's definitely assigned:

```
</> C#
```

```
Console.WriteLine(p.X); // 10
```

2. You can use `new`

```
</> C#
```

```
Point p = new Point();
```

This initializes the struct with its **default values**:

```
</> Plain text
```

```
X = 0  
Y = 0
```

Then you can modify it:

```
</> C#
```

```
p.X = 10;  
p.Y = 20;
```

3. If there is a parameterized constructor

Suppose:

```
</> C#
```

```
struct Point  
{  
    public int X;  
    public int Y;
```

```
public Point(int x, int y)
{
    X = x;
    Y = y;
}
```

Then to use that constructor:

</> C#

```
Point p = new Point(10, 20);
```

You need `new` to invoke the constructor.

Simple rule

</> Plain text

```
Point p;
```

➡ Declare a struct variable, but you must initialize it before reading it.

</> Plain text

```
Point p = new Point();
```

➡ Create the default-initialized value.

</> Plain text

```
Point p = new Point(10, 20);
```

➡ Create the value using your constructor.

One important modern C# detail

With structs, you may see:

</> C#

```
Point p = default;
```

This is another way to get the default value:

</> Plain text

```
X = 0
```

```
Y = 0
```

So **new** is not required just to have a struct variable. It is required when you want to invoke a constructor.

static and sealed class can be inherited or what and why?

No — a **sealed** class cannot be inherited, and a **static** class cannot be inherited either. But the reasons are different.

1. sealed class

</> C#

```
sealed class Animal  
{  
}
```

You **cannot** do:

</> C#

```
class Dog : Animal  
{  
}
```

✗ Compile-time error.

The purpose of **sealed** is explicitly to say:

"This class can be used, but nobody can derive another class from it."

Why would you do that?

For example, if you have a class whose behavior you don't want subclasses to modify or extend through inheritance.

</> C#

```
sealed class PaymentProcessor
{
    public void Process()
    {
    }
}
```

You can still create instances:

```
</> C#

var processor = new PaymentProcessor();
```

You just cannot inherit from it.

2. static class

```
</> C#

static class MathHelper
{
    public static int Add(int x, int y)
    {
        return x + y;
    }
}
```

You cannot inherit from it:

```
</> C#

class MyHelper : MathHelper
{
}
```



But there's an additional restriction:

You **cannot create an instance** of a static class:

```
</> C#
```

```
var helper = new MathHelper(); // ❌
```

You use it directly through the class:

</> C#

```
int result = MathHelper.Add(2, 3);
```

Why can't a static class be inherited?

Because a static class is designed to contain **only static members** and has **no instances**.

Inheritance is fundamentally about creating a derived type whose objects are also objects of the base type.

For example:

</> Plain text

```
Animal
  ↑
Dog
```

You can have:

</> C#

```
Animal a = new Dog();
```

But a static class has no instances:

</> Plain text

```
StaticClass
  ↑
Derived?
```

There's no object of the base static class to participate in that normal inheritance relationship.

Important comparison

	class	sealed class	static class
Can instantiate?	✓	✓	✗
Can inherit from it?	✓	✗	✗
Can have instance members?	✓	✓	✗
Can have static members?	✓	✓	✓
Purpose	Normal object	Prevent inheritance	Utility/shared functionality

One subtle point

A **sealed class** can inherit from another class:

</> C#

```
class Animal
{
}

sealed class Dog : Animal
{
}
```

✓ Dog inherits from Animal .

But nobody can inherit from Dog :

</> C#

```
class Puppy : Dog // ✗
{
}
```

So **sealed** means "cannot be a base class," not "cannot inherit from another class."

class itertaors

If you mean **iterators** in C#, they're very important for .NET interviews. Let's build the concept from the beginning.

1. What is an iterator?

An **iterator** is a mechanism that allows an object to produce a sequence of values **one at a time**.

The most common keywords are:

</> C#

```
yield return  
yield break
```

For example:

</> C#

```
IEnumerable<int> GetNumbers()  
{  
    yield return 1;  
    yield return 2;  
    yield return 3;  
}
```

You can then:

</> C#

```
foreach (int number in GetNumbers())  
{  
    Console.WriteLine(number);  
}
```

Output:

</> Plain text

```
1  
2  
3
```

2. Why not just return a List?

You could do:

</> C#

```
List<int> GetNumbers()  
{  
    return new List<int> { 1, 2, 3 };  
}
```

But this creates and returns the **whole collection**.

With an iterator:

```
</> C#  
  
IEnumerable<int> GetNumbers()  
{  
    yield return 1;  
    yield return 2;  
    yield return 3;  
}
```

the values are produced **one at a time as they're requested**.

This is called **deferred/lazy execution**.

3. The important part: **yield return**

Consider:

```
</> C#  
  
IEnumerable<int> GetNumbers()  
{  
    Console.WriteLine("Before 1");  
  
    yield return 1;  
  
    Console.WriteLine("Before 2");  
  
    yield return 2;  
  
    Console.WriteLine("Before 3");  
  
    yield return 3;  
}
```

If you do:

```
</> C#
```

```
var numbers = GetNumbers();
```

the method doesn't simply execute everything immediately like a normal method returning a list.

The iterator executes as the sequence is enumerated.

For example:

```
</> C#
```

```
foreach (var number in numbers)
{
    Console.WriteLine(number);
}
```

Conceptually:

```
</> Plain text
```

```
request first value
    ↓
execute until yield return 1
    ↓
return 1
    ↓
request next value
    ↓
continue from where it stopped
    ↓
return 2
    ↓
...
```

That's the important idea.

4. yield return doesn't end the method

This is a common beginner confusion.

```
</> C#
```

```
IEnumerable<int> GetNumbers()
{
    yield return 1;
```

```
yield return 2;  
yield return 3;  
}
```

You might think:

</> Plain text

```
yield return 1  
    ↓  
method ends
```

But that's not what happens.

Instead:

</> Plain text

```
yield return 1  
    ↓  
pause iterator  
    ↓  
caller receives 1  
    ↓  
caller asks for next  
    ↓  
resume after yield return 1  
    ↓  
yield return 2
```

The iterator **remembers its state**.

5. yield break

yield break means:

Stop producing values.

Example:

</> C#

```
IEnumerable<int> GetNumbers()  
{  
    yield return 1;
```

```
yield return 2;  
  
yield break;  
  
yield return 3;  
}
```

The result is:

</> Plain text

```
1  
2
```

3 will never be produced.

6. IEnumerable<T> and iterators

Usually an iterator method returns:

</> C#

```
IEnumerable<T>
```

For example:

</> C#

```
IEnumerable<int> GetNumbers()
```

`IEnumerable<T>` represents something that can be **enumerated**.

That's why this works:

</> C#

```
foreach (var number in GetNumbers())  
{  
}
```

7. What does foreach actually use?

Under the hood, `foreach` uses an **enumerator**.

Conceptually:

</> Plain text

```
IEnumerable
  ↓
GetEnumerator()
  ↓
IEnumerator
  ↓
MoveNext()
  ↓
Current
```

The important members are:

</> C#

```
MoveNext()
Current
```

For example, conceptually:

</> Plain text

```
MoveNext() → true
Current → 1
```

```
MoveNext() → true
Current → 2
```

```
MoveNext() → true
Current → 3
```

```
MoveNext() → false
```

Then the loop ends.

8. Iterator vs Enumerator

These terms are related but different.

Iterator

The mechanism/method that **produces the sequence**.

</> C#

```
IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
}
```

Enumerator

The object that **tracks the current position** while enumerating.

It exposes things like:

</> Plain text

```
MoveNext()
Current
```

Think:

</> Plain text

```
Iterator
  ↓
produces sequence

Enumerator
  ↓
moves through sequence
```

9. Iterators can be infinite

This is one of the benefits of lazy execution.

</> C#

```
IEnumerable<int> GetNumbers()
{
    int i = 0;

    while (true)
    {
```

```
        yield return i++;  
    }  
}
```

This can theoretically produce:

</> Plain text

```
0  
1  
2  
3  
4  
5  
...
```

forever.

But:

</> C#

```
foreach (var n in GetNumbers())  
{  
    Console.WriteLine(n);  
}
```

would also run forever.

You could take only some values with LINQ:

</> C#

```
var numbers = GetNumbers().Take(5);
```

Result:

</> Plain text

```
0  
1  
2  
3  
4
```

Only those requested values need to be generated.

10. Important: deferred execution

This is a major interview concept.

```
</> C#
```

```
IEnumerable<int> GetNumbers()  
{  
    Console.WriteLine("Executed");  
    yield return 1;  
}
```

Then:

```
</> C#
```

```
var numbers = GetNumbers();
```

Does "Executed" necessarily print here?

No.

The sequence hasn't necessarily been enumerated yet.

When you do:

```
</> C#
```

```
foreach (var n in numbers)  
{  
}
```

the iterator starts executing.

This is called **deferred execution**.

11. Why are iterators useful?

They are useful when:

Large collections

You don't need to create the entire result in memory.

Expensive calculations

You calculate each value only when requested.

Filtering

You can generate only values that satisfy some condition.

Streaming

You can process data progressively.

Infinite sequences

You can represent sequences that don't have a natural end.

12. Iterator vs returning `List<T>`

Consider:

`</> C#`

```
List<int> GetNumbers()
{
    var result = new List<int>();

    for (int i = 0; i < 1_000_000; i++)
    {
        result.Add(i);
    }

    return result;
}
```

This builds the whole list first.

Iterator:

`</> C#`

```
IEnumerable<int> GetNumbers()
{
    for (int i = 0; i < 1_000_000; i++)
    {
        yield return i;
    }
}
```

```
}  
}
```

The second version can produce numbers **as they're requested**.

For example:

```
</> C#
```

```
GetNumbers().Take(10);
```

doesn't need to generate all one million values.

13. Very important interview distinction

Don't say:

"IEnumerable always loads everything into memory."

That's incorrect.

For example:

```
</> C#
```

```
IEnumerable<int> numbers = GetNumbers();
```

can represent a lazy iterator.

But if you do:

```
</> C#
```

```
var list = numbers.ToList();
```

then the sequence is enumerated and the results are stored in a list.

Similarly:

```
</> C#
```

```
numbers.ToArray();
```

materializes the sequence into an array.

So:

</> Plain text

```
IEnumerable
  ↓
potentially lazy

ToList()
  ↓
materialize everything
```

14. yield return and return

A method using `yield return` behaves differently from a normal method.

Normal:

</> C#

```
List<int> GetNumbers()
{
    return new List<int> { 1, 2, 3 };
}
```

The method executes and returns the complete object.

Iterator:

</> C#

```
IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

The compiler transforms the iterator method into a state-machine-like implementation that remembers where execution should resume.

You don't normally write that machinery yourself.

15. Interview question: Can an iterator method use return ?

You cannot use a normal value-returning `return` in an iterator method.

For example:

`</> C#`

```
IEnumerable<int> GetNumbers()  
{  
    yield return 1;  
  
    return 5; // ❌  
}
```

But you can use:

`</> C#`

```
yield break;
```

to stop iteration.

16. The mental model

Think about an iterator like a **vending machine**.

Instead of giving you the entire collection:

`</> Plain text`

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

it gives you:

`</> Plain text`

```
Give me next
```

```
↓
```

```
1
```

```
Give me next
```

```
↓
```

```
2
```

```
Give me next
```

```
↓
```

```
3
```

The iterator remembers where it stopped.

That's exactly what makes:

```
</> C#
```

```
yield return
```

so useful.

The interview essentials to memorize

```
</> Plain text
```

```
Iterator
```

```
↓
```

```
Produces values one at a time
```

```
yield return
```

```
↓
```

```
Returns one value and pauses
```

```
yield break
```

```
↓
```

```
Stops iteration
```

```
IEnumerable<T>
```

```
↓
```

```
Represents an enumerable sequence
```

```
IEnumerator<T>
```

```
↓
```

```
Tracks the current position
```

If you're learning this for **C# interviews**, the next closely related topic is **IEnumerable** vs **IEnumerator** vs **ICollection** vs **IList** vs **IQueryable** , because interviewers commonly ask them together.

yes go

Yes. Let's connect them in a way that's useful for interviews.

1. The hierarchy

The easiest way to understand them is:

</> Plain text

```
IEnumerable<T>
  ↑
ICollection<T>
  ↑
IList<T>
```

And separately:

</> Plain text

```
IEnumerable<T>
  ↑
IQueryable<T>
```

`IList<T>` inherits from `ICollection<T>`, and `ICollection<T>` / `IList<T>` ultimately provide enumeration through `IEnumerable<T>`. [🌐 Microsoft Learn +1](#)

2. `IEnumerable<T>` — "I can enumerate this"

This is the most basic interface.

</> C#

```
IEnumerable<int> numbers = ...;
```

Its main purpose is:

Give me the elements one by one.

It provides `GetEnumerator()`. [🔗 GitHub](#)

So:

</> C#

```
foreach (var number in numbers)
{
    Console.WriteLine(number);
}
```

works.

Important

`IEnumerable<T>` does **not** promise that you can:

- add
- remove
- modify
- access by index

It's basically:

</> Plain text

"I can give you my elements."

3. `IEnumerator<T>` — "I am the thing moving through them"

Remember the iterator discussion?

`IEnumerable<T>` gives you an enumerator:

</> Plain text

```
IEnumerable<T>
    ↓
GetEnumerator()
    ↓
IEnumerator<T>
```

The enumerator tracks where you currently are.

Conceptually:

</> Plain text

Collection:
[10, 20, 30]

Enumerator:

↓
10

Then:

</> Plain text

MoveNext()

↓
20

MoveNext()

↓
30

MoveNext()

↓
false

So:

IEnumerable<T>

"I can be enumerated."

IEnumerator<T>

"I am currently doing the enumeration."

That's why you normally use:

</> C#

foreach

instead of manually using `IEnumerator` .

4. **ICollection<T>** — "I am a collection"

Now we add collection-management functionality.

</> C#

`ICollection<int> numbers = ...;`

You get things such as:

</> Plain text

```
Count
Add()
Remove()
Clear()
Contains()
CopyTo()
```

So the progression is:

</> Plain text

```
IEnumerable<T>
  ↓
Can enumerate

ICollection<T>
  ↓
Can enumerate
+
Can manage the collection
```

Microsoft defines `IList<T>` as a descendant of `ICollection<T>`, with operations such as `Add`, `Remove`, `Insert`, and indexed access. [🌐 Microsoft Learn](#)

5. `IList<T>` — "I also have indexes"

Now we add index-based access.

</> C#

```
IList<int> numbers = new List<int>();
```

You can do:

</> C#

```
numbers[0]
numbers[1]
numbers[2]
```

And you get:

</> Plain text

```
Add()
Remove()
Clear()
Contains()
Count
+
IndexOf()
Insert()
RemoveAt()
[index]
```

So:

</> Plain text

```
IEnumerable<T>
  |
  | enumerate
  ↓
ICollection<T>
  |
  | add/remove/count
  ↓
IList<T>
  |
  | index/insert/remove-at
  ↓
```

6. Real example with List<T>

List<T> implements these interfaces.

So:

</> C#

```
List<int> list = new List<int>();
```

You can assign it to:

</> C#

```
IEnumerable<int> a = list;  
ICollection<int> b = list;  
IList<int> c = list;
```

But notice what happens to the available operations.

As `List<T>`

```
</> C#  
  
list.Add(10);  
list.Remove(10);  
list[0];
```

As `IEnumerable<T>`

```
</> C#  
  
a
```

You primarily treat it as:

```
</> Plain text  
  
"something I can enumerate"
```

You don't get:

```
</> C#  
  
a.Add(10); // ✖  
a[0];      // ✖
```

The **underlying object** hasn't changed. Only the interface through which you're viewing it has changed.

This is very important in .NET.

7. Why would I use `IEnumerable<T>` instead of `List<T>` ?

Suppose you have:

```
</> C#
```

```
List<Customer> customers = ...
```

and your method only needs to **read/enumerate** customers.

Instead of:

```
</> C#
```

```
void ProcessCustomers(List<Customer> customers)
```

you could use:

```
</> C#
```

```
void ProcessCustomers(IEnumerable<Customer> customers)
```

Now the method doesn't care whether the caller gives it:

```
</> Plain text
```

```
List<Customer>  
Array  
HashSet<Customer>  
LINQ query  
Iterator  
etc.
```

It only requires:

"Give me something I can enumerate."

This makes the method less coupled to a specific collection type.

8. **IQueryable<T>** — this one is different

This is especially important for you because you're working with **EF Core**.

You might have:

```
</> C#
```

```
IQueryable<Customer> customers = _dbContext.Customers;
```

`IQueryable<T>` represents a query that can be understood by a **query provider**.

It has things like:

`</>` Plain text

Expression

Provider

ElementType

and it inherits from `IEnumerable`, allowing the results to eventually be enumerated. [Microsoft Learn](#)

The big difference is **where the query executes**.

9. IEnumerable vs IQueryable

This is a very common interview question.

Suppose:

`</>` C#

```
_dbContext.Customers
```

is an `IQueryable<Customer>`.

Then:

`</>` C#

```
var result = _dbContext.Customers
    .Where(x => x.Age > 18)
    .Select(x => x.Name);
```

EF Core can translate that expression into a database query.

Conceptually:

`</>` Plain text

C# LINQ
↓
Expression tree
↓
EF Core
↓
SQL
↓
Database

But after you materialize the data:

```
</> C#  
  
var customers = await _dbContext.Customers.ToListAsync();
```

you now have a `List<Customer>` .

If you then do:

```
</> C#  
  
customers.Where(x => x.Age > 18)
```

the filtering happens in **memory**.

Conceptually:

```
</> Plain text  
  
Database  
↓  
all customers  
↓  
List<Customer>  
↓  
LINQ  
↓  
filter in memory
```

That's the key difference.

Microsoft's documentation notes that enumerating an `IQueryable` causes its expression tree to be executed, with the specific execution determined by the query provider; a provider such as an ORM can translate the expression for its underlying data source. [Microsoft Learn](#)

10. Very important: IEnumerable doesn't always mean "in memory"

Be careful with this interview statement:

"IEnumerable means in-memory."

That's an oversimplification.

`IEnumerable<T>` describes **enumeration**.

It doesn't inherently say where the data originated.

The more useful distinction with EF Core is:

`</>` Plain text

`IQueryable`

→ query can be translated by the provider

`IEnumerable`

→ you're working with an enumerable sequence

Once you call:

`</>` C#

`ToList()`

`ToArray()`

`ToListAsync()`

`ToArrayAsync()`

you materialize the results.

11. Example you should know for interviews

Query before materialization

`</>` C#

```
var query = _dbContext.Customers
    .Where(x => x.Age > 18);
```

Still:

</> Plain text

```
IQueryable<Customer>
```

The query hasn't necessarily been executed yet.

Then:

</> C#

```
var result = await query.ToListAsync();
```

Now the database query executes and the results are loaded.

After materialization

</> C#

```
var result = await _dbContext.Customers
    .ToListAsync();

var adults = result
    .Where(x => x.Age > 18);
```

Now:

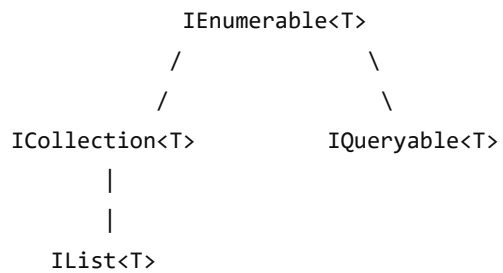
</> Plain text

```
Database
  ↓
List<Customer>
  ↓
Where()
  ↓
in-memory filtering
```

12. The full picture

Memorize this:

</> Plain text



IEnumerable<T>

Can enumerate

</> Plain text

foreach
GetEnumerator

ICollection<T>

Can manage a collection

</> Plain text

Count
Add
Remove
Clear
Contains

IList<T>

Can work with indexes

</> Plain text

[0]
Insert
RemoveAt
IndexOf

IQueryable<T>

Represents a provider-backed query

</> Plain text

```

Expression
Provider
  ↓
Query provider
  ↓
Database / other data source

```

Interview answer

If the interviewer asks:

"What's the difference between IEnumerable, ICollection, IList and IQueryable?"

A good concise answer is:

" IEnumerable<T> provides enumeration. ICollection<T> adds collection-management operations such as Add, Remove and Count. IList<T> additionally provides index-based access and insertion/removal by index. IQueryable<T> is designed for provider-backed queries; it exposes an expression tree and query provider, so a provider such as EF Core can translate the query for the underlying data source."

That's a strong interview answer.

but if a class has a list and i wanna accesss iterate this list using class index like that car[2]

Yes — a class can contain a List<T> and you can make the class itself support index access like:

```
</> C#
```

```
car[2]
```

But you need to define an **indexer** in the class.

Example

```
</> C#
```

```

class Car
{
    private List<string> parts = new List<string>
    {
        "Engine",
        "Wheel",

```

```
        "Door",  
        "Seat"  
    };  
  
    public string this[int index]  
    {  
        get { return parts[index]; }  
        set { parts[index] = value; }  
    }  
}
```

Now:

```
</> C#  
  
Car car = new Car();  
  
Console.WriteLine(car[2]);
```

Output:

```
</> Plain text  
  
Door
```

And you can also assign:

```
</> C#  
  
car[2] = "Window";
```

What is happening?

When you write:

```
</> C#  
  
car[2]
```

C# translates it conceptually to:

```
</> C#  
  
car.this[2]
```

which calls your indexer's `get` :

```
</> C#  
  
get { return parts[index]; }
```

So the **indexer** allows an object of your class to behave like an indexed collection.

Very important distinction

If you simply have:

```
</> C#  
  
class Car  
{  
    public List<string> Parts { get; set; }  
}
```

You **cannot** do:

```
</> C#  
  
car[2] ❌
```

You would have to do:

```
</> C#  
  
car.Parts[2] ✅
```

Unless you add an **indexer**:

```
</> C#  
  
public string this[int index] => Parts[index];
```

Then:

```
</> C#  
  
car[2] ✅
```

This is called a **C# indexer**.

Today 7:41 PM

what ienumerable inherit from

`IEnumerable<T>` inherits from `IEnumerable` .

```
</> C#  
  
IEnumerable<T>  
    ↓  
IEnumerable
```

More precisely:

```
</> C#  
  
public interface IEnumerable<out T> : IEnumerable
```

What each one provides

`IEnumerable` provides:

```
</> C#  
  
IEnumerator GetEnumerator();
```

`IEnumerable<T>` adds the strongly typed version:

```
</> C#  
  
IEnumerator<T> GetEnumerator();
```

So:

```
</> C#  
  
IEnumerable<string> names;
```

means you can iterate through it with:

```
</> C#
```

```
foreach (string name in names)
{
    // ...
}
```

Related hierarchy

</> Plain text

```
IEnumerable
  ↑
IEnumerable<T>
  ↑
ICollection<T>
  ↑
IList<T>
```

For example:

</> C#

```
List<int>
```

implements `IList<int>` , which inherits from `ICollection<int>` , which inherits from `IEnumerable<int>` .

Interview answer:

`IEnumerable<T>` inherits from the non-generic `IEnumerable` interface and provides strongly typed enumeration through `IEnumerator<T>` .

what `IEnumerable` inherit till object

If you mean **the inheritance chain from `IEnumerable` up to `object`** , there is an important point:

`IEnumerable` does NOT inherit from `object` directly

Interfaces don't inherit from `object` .

</> Plain text

```
object
  ↑
  └─ no inheritance relationship with IEnumerable

IEnumerable
```

```

↑
IEnumerable<T>
↑
ICollection<T>
↑
IList<T>
↑
List<T> ← class

```

`List<T>` is a **class**, so it ultimately inherits from `object` :

</> Plain text

```

object
↑
IEnumerable
↑
IEnumerable<T>
↑
ICollection<T>
↑
IList<T>
↑
List<T>

```

But that diagram can be misleading because the arrows between `object` and the interfaces aren't inheritance arrows.

The technically correct picture is:

</> Plain text

```

object
↑
List<T>
├── implements IList<T>
│   └── inherits ICollection<T>
│       └── inherits IEnumerable<T>
│           └── inherits IEnumerable
└── (other interfaces...)

```

Why?

`object` is the base class of **all classes**.

`IEnumerable` is an **interface**, not a class. It defines a contract for enumeration and doesn't have `object` as its base interface.

So if an interviewer asks:

"What does IEnumerable inherit from?"

Answer:

IEnumerable doesn't inherit from another interface; it's the base non-generic enumeration interface. **IEnumerable<T>** inherits from **IEnumerable** .